CĐ-07 · NumPy và độ phức tạp thuật toán

Luyện kiến thức: không tính giờ, chọn xong là biết đúng/sai và xem giải thích. Tiến độ lưu riêng với Luyện đề.

Mã đề **PETER-CD07-0718** · Người làm: Peter: gửi mã này cho mentor khi chữa bài.

Phần A — Nhận biết

Câu 1 · T07-A03 · 3 điểm · Trắc nghiệm · Nhận biết

Cho `a` và `b` là hai mảng NumPy 1 chiều cùng shape `(n,)`, và `c = np.empty(n)`. Vòng lặp sau tương đương với biểu thức vector hoá nào?

```
for i in range(n):
    c[i] = a[i] * b[i]
```

- ○ **A.** `c = np.outer(a, b)`
- ○ **B.** `c = a * b`
- ○ **C.** `c = a @ b`
- ○ **D.** `c = np.prod([a, b])`

Câu 2 · T07-A05 · 3 điểm · Trắc nghiệm · Nhận biết

Theo bảng TimeComplexity của CPython, với `lst` là list n phần tử, `s` là set và `d` là dict (mỗi cái n phần tử), thao tác nào có chi phí **trung bình** $O(n)$?

- ○ **A.** `lst.pop(0)`
- ○ **B.** `d[k] = v`
- ○ **C.** `lst.append(x)`
- ○ **D.** `x in s`

Câu 3 · T07-A06 · 3,5 điểm · Điền số · Nhận biết

Đoạn code sau in ra giá trị nào (cũng chính là số lần lệnh `dem += 1` được thực thi)? **Nhập số nguyên.**

```
dem = 0
for i in range(12):
    for j in range(i, 12):
        dem += 1
print(dem)
```

Nhập số

Kiểm tra

Câu 4 · T07-A07 · 3 điểm · Trắc nghiệm · Nhận biết

Phân tích một thuật toán cho số phép toán $T(n) = 4n\log_2 n + 0{,}5\,n^{1{,}5} + 200$. Cận Big-O **chặt nhất** (dạng gọn) của $T(n)$ là gì?

- ○ **A.** $O(n \log n)$
- ○ **B.** $O(n^2)$
- ○ **C.** $O(n^{1{,}5})$
- ○ **D.** $O(n^{1{,}5} \log n)$

Câu 5 · T07-A02 · 3,5 điểm · Điền kết quả · Nhận biết

Mảng `a` ghi điểm của 3 mô hình (hàng) trên 2 bộ test (cột). Đoạn code sau in ra gì? Ghi đúng từng dòng.

```
import numpy as np
a = np.array([[4, 9],
              [7, 2],
              [5, 6]])
print(a.max(axis=0).tolist())
print(a.argmax(axis=1).tolist())
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 6 · T07-A08 · 3 điểm · Trắc nghiệm · Nhận biết

Tìm kiếm nhị phân trên mảng **đã sắp xếp** gồm $n = 10^6$ phần tử; mỗi vòng lặp so sánh với phần tử giữa rồi bỏ đi một nửa khoảng. Trong trường hợp xấu nhất cần khoảng bao nhiêu vòng lặp?

- ○ **A.** Khoảng 1000 vòng
- ○ **B.** Khoảng 20 vòng
- ○ **C.** Khoảng 6 vòng
- ○ **D.** Khoảng $500\,000$ vòng

Câu 7 · T07-A01 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Một batch embedding gồm 32 câu, mỗi câu được mã hoá thành vector 768 chiều và lưu trong mảng NumPy `E` với `E.shape == (32, 768)`. Giá trị của `E.ndim` và `E.size` lần lượt là gì?

- ○ **A.** `2` và `24576`
- ○ **B.** `32` và `24576`
- ○ **C.** `2` và `800`
- ○ **D.** `1` và `24576`

Câu 8 · T07-A04 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Mảng `scores` shape `(n,)` chứa xác suất mô hình phân loại spam gán cho n email. Biểu thức vector hoá nào **đếm đúng** số email có điểm lớn hơn 0,8?

- ○ **A.** `np.count_nonzero(scores) > 0.8`
- ○ **B.** `(scores > 0.8).sum()`
- ○ **C.** `len(scores > 0.8)`

- D. `np.sum(scores[scores > 0.8])`

Phần B — Thông hiểu

Câu 9 · T07-B08 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Luồng điểm số từ mô hình xếp hạng đi qua đoạn code giữ lại 3 điểm bằng `heapq`. Đoạn code in ra gì?

```
import heapq
h = []
for x in [5, 1, 8, 3, 9, 2]:
    heapq.heappush(h, x)
    if len(h) > 3:
        heapq.heappop(h)
print(sorted(h), h[0])
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 10 · T07-B06 · 3 điểm · Trắc nghiệm · Thông hiểu

Xét hàm đệ quy sau (giả sử n nhỏ, không chạm giới hạn đệ quy):

```
def f(n):
    if n == 0:
        return 1
    return f(n - 1) + f(n - 1)
```

Tổng số lần gọi `f` khi tính `f(n)` tăng theo bậc nào?

- A. $O(n \log n)$
- B. $O(n^2)$
- C. $O(n)$
- D. $O(2^n)$

Câu 11 · T07-B07 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Dict `users` ánh xạ `user_id` sang email cho n người dùng. Khi nhập m email đăng ký mới, code kiểm tra trùng bằng `if email in users.values(): ...` cho từng email. Đánh giá nào đúng?

- ○ **A.** Tổng $O(m \log n)$ vì dict lưu các khóa theo thứ tự đã sắp.
- ○ **B.** Tổng $O(m)$ vì tra cứu trong dict luôn chỉ tốn $O(1)$.
- ○ **C.** Tổng $O(m \cdot n)$; nên đổi sang `email in sorted(users.values())`.
- ○ **D.** Tổng $O(m \cdot n)$; nên dựng `set(users.values())` một lần rồi tra, tổng $O(n + m)$.

Câu 12 · T07-B01 · 3 điểm · Trắc nghiệm · Thông hiểu

Đoạn code sau in ra gì?

```
import numpy as np
a = np.arange(5)
b = a[1:4]
b[0] = 100
c = a[[0, 2]]
c[0] = -1
print(a.tolist())
```

- ○ **A.** `[0, 100, 2, 3, 4]`
- ○ **B.** `[-1, 1, 2, 3, 4]`
- ○ **C.** `[-1, 100, 2, 3, 4]`
- ○ **D.** `[0, 1, 2, 3, 4]`

Câu 13 · T07-B05 · 3 điểm · Trắc nghiệm · Thông hiểu

Bạn có hàm Python thuần `f(x)` xử lý một số vô hướng, rồi viết `vf = np.vectorize(f)` và gọi `vf(arr)` trên mảng 1 triệu phần tử. Phát biểu nào **đúng**?

- ○ **A.** Nó làm độ phức tạp giảm từ $O(n)$ xuống $O(1)$ với n phần tử.
- ○ **B.** Nó dịch `f` sang mã C nên chạy nhanh ngang các ufunc có sẵn như `np.exp`.
- ○ **C.** Nó tự chia các lần gọi `f` ra nhiều lõi CPU để chạy song song.

- **D.** Nó tiện cho broadcasting, nhưng vẫn gọi `f` lần lượt cho từng phần tử.

Câu 14 · T07-B10 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Danh sách `ids` gồm n mã tài liệu (chuỗi) bị trùng do crawler chạy lặp. Cần khử trùng lặp nhưng **giữ thứ tự xuất hiện đầu tiên**, thời gian trung bình $O(n)$. Biểu thức nào đáp ứng cả hai yêu cầu?

- **A.** `[x for i, x in enumerate(ids) if x not in ids[:i]]`
- **B.** `list(dict.fromkeys(ids))`
- **C.** `sorted(set(ids))`
- **D.** `list(set(ids))`

Câu 15 · T07-B03 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Bảng `x` đếm lượt click của 3 sản phẩm (hàng) trên 2 kênh web và app (cột). Cần tỉ lệ đóng góp của từng kênh **trong mỗi sản phẩm**. Đoạn code sau in ra gì?

```
import numpy as np
x = np.array([[1., 3.],
              [2., 6.],
              [0., 5.]])
m = x.sum(axis=1, keepdims=True)
p = x / m
print(m.shape, p[:, 0].tolist())
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 16 · T07-B04 · 3,5 điểm · Điền số · Thông hiểu · tình huống

Một hệ thống RAG lưu chỉ mục vector gồm 5×10^5 đoạn văn, mỗi đoạn một embedding 768 chiều, trong một mảng NumPy liên tục `E` kiểu `float32`. Thuộc tính `E.nbytes` tương ứng bao nhiêu GB? (Quy ước 1 GB $= 10^9$ byte; **làm tròn 3 chữ số thập phân**, chỉ nhập số, không kèm đơn vị.)

Nhập số

Kiểm tra

Câu 17 · T07-B09 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Bộ xếp hạng trả về list Python `scores` gồm $n = 10^7$ điểm; cần lấy 10 tài liệu điểm cao nhất bằng `heapq.nlargest(10, scores)`. Gọi $k = 10$. Độ phức tạp thời gian của cách này là gì?

- A. $O(n \cdot k)$
- B. $O(k \log n)$
- C. $O(n \log k)$
- D. $O(n \log n)$

Câu 18 · T07-B02 · 3 điểm · Trắc nghiệm · Thông hiểu

Với `X = np.ones(sx)` và `Y = np.ones(sy)`, cặp shape `sx`, `sy` nào khiến `X + Y` báo lỗi `ValueError` vì không broadcast được?

- A. `(2, 3, 4)` và `(3, 1)`
- B. `(4, 1, 3)` và `(5, 1)`
- C. `(32, 1)` và `(1, 768)`
- D. `(32, 768)` và `(32,)`

Phần C — Vận dụng

Câu 19 · T07-C06 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Dịch vụ giám sát nhận n bản ghi log liên tục và cần danh sách `recent` với bản ghi **mới nhất ở đầu**. Code hiện tại:

```
recent = []
for r in stream:
    recent.insert(0, r)
```

Độ phức tạp tổng và cách sửa nào đúng?

- ○ **A.** $O(n^2)$; dùng `collections.deque` với `appendleft`, mỗi lần $O(1)$.
- ○ **B.** $O(n \log n)$; thay bằng `heapq.heappush` để bản ghi mới lên đầu.
- ○ **C.** $O(n)$; giữ nguyên vì chèn vào list luôn tốn $O(1)$.
- ○ **D.** $O(n^2)$; viết lại thành `recent = [r] + recent` cho gọn.

Câu 20 · T07-C09 · 6 điểm · Code viết tay · Vận dụng · tình huống

Bối cảnh. Sàn thương mại điện tử muốn gợi ý combo 2 sản phẩm có tổng giá **đúng bằng** ngân sách `k` (nghìn đồng). Viết hàm Python `cap_tong_k(gia, k)`:

- `gia`: list số nguyên dương (có thể trùng giá), độ dài n có thể tới 10^6.
- Trả về list các cặp **giá trị** `(a, b)` với `a <= b` và `a + b == k`, hai phần tử lấy từ **hai vị trí khác nhau** trong `gia`; mỗi cặp giá trị xuất hiện đúng một lần; list sắp tăng dần theo `a`.
- Ví dụ: `cap_tong_k([3, 5, 2, 5, 8, 7], 10)` trả `[(2, 8), (3, 7), (5, 5)]`; `cap_tong_k([5, 1], 10)` trả `[]`.

Yêu cầu: không dùng hai vòng lặp lồng nhau trên toàn danh sách; ghi rõ độ phức tạp thời gian và bộ nhớ của lời giải (trong docstring hoặc chú thích).

```
Viết code Python như trên giấy (không chạy được)…
```

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 21 · T07-C01 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

`Q` có shape `(8, 384)` (8 câu hỏi) và `D` có shape `(1000, 384)` (1000 đoạn văn); cả hai đã chuẩn hoá L2 theo hàng nên `S = Q @ D.T` là ma trận cosine. Để lấy **chỉ số** đoạn văn giống nhất cho **từng câu hỏi**, dùng biểu thức nào và kết quả có shape gì?

- ○ **A.** `S.argmax(axis=1)`, shape `(8,)`
- ○ **B.** `S.argmax()`, shape `(8,)`
- ○ **C.** `S.max(axis=1)`, shape `(8,)`

○ **D.** `S.argmax(axis=0)`, shape `(1000,)`

Câu 22 · T07-C08 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Thực đơn có giá các món (chục nghìn đồng) đã sắp tăng dần. Hàm sau tìm hai món có tổng đúng bằng ngân sách `k` bằng kỹ thuật hai con trỏ. Dòng cuối in ra gì?

```
def tim_cap(a, k):
    l, r, buoc = 0, len(a) - 1, 0
    while l < r:
        buoc += 1
        s = a[l] + a[r]
        if s == k:
            return (a[l], a[r]), buoc
        if s < k:
            l += 1
        else:
            r -= 1
    return None, buoc

print(tim_cap([1, 3, 4, 6, 8, 11], 10))
```

○ **A.** `(None, 5)`

○ **B.** `((4, 6), 5)`

○ **C.** `((4, 6), 4)`

○ **D.** `((4, 6), 6)`

Câu 23 · T07-C02 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Batch ảnh `X` đọc bằng OpenCV có shape `(N, H, W, C) = (16, 224, 224, 3)` (kênh màu ở cuối). Mô hình cần đầu vào dạng `(N, C, H, W)` và mỗi ảnh phải **giữ nguyên nội dung** (không bị xoay, lật hay xáo pixel). Biểu thức nào đúng?

○ **A.** `X.transpose(0, 2, 3, 1)`

○ **B.** `X.transpose(0, 3, 1, 2)`

○ **C.** `X.reshape(16, 3, 224, 224)`

○ **D.** `np.swapaxes(X, 1, 3)`

Câu 24 · T07-C07 · 3 điểm · Trắc nghiệm · Vận dụng

Với list `a` có n phần tử (n đủ nhỏ để không vượt giới hạn đệ quy), độ phức tạp thời gian của `tong(a)` là gì?

```
def tong(a):
    if not a:
        return 0
    return a[0] + tong(a[1:])
```

- A. $O(n^2)$
- B. $O(n)$
- C. $O(n \log n)$
- D. $O(2^n)$

Câu 25 · T07-C04 · 3,5 điểm · Điền số · Vận dụng · tình huống

Một bạn tính khoảng cách Euclid bình phương giữa 2000 vector truy vấn `Q` (shape `(2000, 128)`) và 5000 vector trong kho `K` (shape `(5000, 128)`), cả hai kiểu `float32`, bằng broadcasting:

```
diff = Q[:, None, :] - K[None, :, :]
D2 = (diff ** 2).sum(axis=2)
```

Riêng mảng trung gian `diff` chiếm bao nhiêu GB? (Quy ước 1 GB $= 10^9$ byte; **làm tròn 2 chữ số thập phân**, chỉ nhập số.)

Nhập số | Kiểm tra

Câu 26 · T07-C03 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Mảng `req` là số request mỗi phút của một API. Đoạn code tính tổng theo cửa sổ trượt 3 phút **không dùng vòng lặp** bằng tổng tiền tố (prefix sum). Đoạn code in ra gì?

```
import numpy as np
req = np.array([4, 2, 6, 1, 3, 7])
c = np.concatenate(([0], np.cumsum(req)))
w = c[3:] - c[:-3]
print(w.tolist(), int(w.argmax()))
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 27 · T07-C05 · 3 điểm · Trắc nghiệm · Vận dụng

Độ phức tạp thời gian của hàm sau theo n là gì?

```
def dem(n):
    c = 0
    for i in range(1, n + 1):
        for j in range(0, n, i):
            c += 1
    return c
```

- A. $O(n^2)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(n\sqrt{n})$

Phần D — Vận dụng cao

Câu 28 · T07-D02 · 3 điểm · Trắc nghiệm · Vận dụng cao · tình huống

Trang danh mục cần hiển thị sản phẩm theo **danh mục tăng dần**, trong cùng danh mục thì **giá giảm dần**; sản phẩm cùng danh mục và cùng giá giữ thứ tự gốc. Một lập trình viên dùng hai lần sắp xếp:

```
rows = [("ao", "A", 200), ("but", "B", 50), ("mu", "A", 150),
        ("tui", "B", 180), ("giay", "A", 200), ("vi", "B", 50)]
rows.sort(key=lambda r: r[2], reverse=True)
rows.sort(key=lambda r: r[1])
print([r[0] for r in rows])
```

Dòng `print` in ra gì?

- A. `['ao', 'giay', 'tui', 'mu', 'but', 'vi']`

- B. `['giay', 'ao', 'mu', 'tui', 'vi', 'but']`
- C. `['ao', 'mu', 'giay', 'but', 'tui', 'vi']`
- D. `['ao', 'giay', 'mu', 'tui', 'but', 'vi']`

Câu 29 · T07-D03 · 6 điểm · Code viết tay · Vận dụng cao · tình huống

Bối cảnh. Hệ thống tìm kiếm ngữ nghĩa có ma trận embedding câu hỏi `Q` shape `(m, d)` và kho đoạn văn `D` shape `(n, d)` (số thực). Viết hàm NumPy `topk_cosine(Q, D, k)` trả về mảng số nguyên shape `(m, min(k, n))` : hàng i là chỉ số các đoạn văn có cosine similarity với `Q[i]` lớn nhất, **xếp giảm dần** theo cosine, với

$$\cos(\mathbf{q}, \mathbf{x}) = \frac{\mathbf{q} \cdot \mathbf{x}}{\|\mathbf{q}\| \, \|\mathbf{x}\|}.$$

Yêu cầu:

- Không dùng vòng lặp Python theo m hoặc n (dùng broadcasting và phép nhân ma trận `@`).
- Vector có chuẩn bằng 0 thì coi cosine của nó với mọi vector là 0 (không được sinh `nan`).
- Không sắp xếp toàn bộ mỗi hàng: dùng `np.argpartition` để chọn k phần tử, rồi chỉ sắp k phần tử đó.
- Ghi rõ độ phức tạp thời gian và bộ nhớ theo m, n, d, k.

Có thể giả định trong cùng một hàng không có hai cosine bằng nhau (trừ các cosine bằng 0 do vector 0).

Viết code Python như trên giấy (không chạy được)…

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 30 · T07-D01 · 3,5 điểm · Điền số · Vận dụng cao · tình huống

Một buffer tự viết để gom embedding hoạt động như mảng động: dung lượng ban đầu là 1; khi append mà buffer **đã đầy**, nó cấp vùng nhớ mới **gấp đôi** dung lượng, **sao chép toàn bộ phần tử cũ** sang, rồi mới ghi phần tử mới. Bắt đầu từ buffer rỗng, append liên tiếp 100 phần tử. Tổng số lần sao chép phần tử cũ (không tính 100 lần ghi phần tử mới) là bao nhiêu? **Nhập số nguyên.**

Nhập số

Kiểm tra